NL2SQL System Architecture & Codebase Audit

1. Executive Summary

The NL2SQL (Natural Language to SQL) application is a full-stack, AI-powered assistant designed to bridge the gap between non-technical users and complex relational databases. The core problem it solves is the inability of business users to extract data from databases without knowing SQL syntax.

By uploading an SQLite database file to the web interface, users can ask questions in plain English. The system intelligently converts these questions into valid, secure SQL queries, executes them against the uploaded database, and returns both the query and the resulting data in an intuitive chat interface. It utilizes a prioritized pipeline that relies on a local T5 Machine Learning model specifically fine-tuned for SQL generation, falling back to the Gemini API only when the model's confidence is low or when complex query repair is necessary.

---

2. Technology Stack

The application is built on a modern, modular architecture leveraging the following core libraries and frameworks:

Frontend

- **React (Vite):** Provides a fast, component-based user interface. Vite is used for rapid development and optimized production builds.
- **Tailwind CSS (v4):** Utilized for rapid, utility-first UI styling. Provides the dark mode, modern chat interface, and responsive design.
- **Framer Motion:** Adds micro-animations and smooth transitions (e.g., message bubbles popping in, loading indicators) to create a premium, "living" application feel.
- **Axios:** Handles asynchronous HTTP requests to the Flask backend for uploading files and fetching query results.

Backend & AI Inference

- **Python (Flask):** The lightweight web framework routing HTTP requests, handling CORS, and orchestrating the AI pipelines.
- **SQLite (sqlite3):** The target database engine. Native Python support allows seamless file parsing, schema extraction, and query execution.
- **Hugging Face Transformers (`transformers`):** Loads and runs the primary local NLP model (`tscholak/3vnuv1vf` - T5 Large). It handles the beam search decoding for accurate SQL generation.
- **PyTorch (`torch`):** The underlying tensor computation framework powering the local T5 model.
- **Google GenAI SDK (`google-genai`):** Integrates the Gemini 1.5 Pro API. Serves as a highly capable fallback engine for edge cases, low-confidence T5 outputs, or error repairs.

---

3. File-by-File Directory

Root Level

File/Directory	Description
`frontend/`	Contains the entire React, Vite, and Tailwind CSS web application.
`backend/`	Contains the Flask API, AI inference logic, and database operations.
`requirements.txt`	Defines all Python dependencies required to run the backend server.
`create_sample_db.py`	A utility script used to generate a small testing database (`sample_school.db`).

Frontend Directory (`/frontend`)

File	Description
`src/App.jsx`	The main React application wrapper. Manages the high-level state (connected database, view switching).
`src/components/FileUpload.jsx`	UI component handling drag-and-drop database uploads, file size validation, and the `POST /upload_db` API call.
`src/components/ChatComponent.jsx`	The core conversational UI. Renders messages, SQL code blocks (with syntax highlighting), result tables, and handles the `POST /api/query` API call.
`src/index.css`	Global stylesheet integrating Tailwind CSS v4 and custom base styles.

File	Description
`postcss.config.js`	Configures PostCSS to use the `@tailwindcss/postcss` plugin required for Tailwind v4.

Backend Directory (`/backend`)

File	Description
`app.py`	The main Flask entry point. Defines API routes (`/upload_db` , `/api/query`), configures CORS, and orchestrates the rule engine, model inference, and Gemini fallback pipeline.
`config/settings.py`	Global configuration variables including local file paths, model selection (`tscholak/3vnuv1vf`), device mapping (CPU/GPU), and sequence length limits. Reads from `last_db.txt` for persistence.
`model/load_model.py`	Handles the secure downloading and loading of the Hugging Face model and tokenizer. Includes a multi-tier loading strategy prioritizing `safetensors` .
`model/inference.py`	Exposes the `generate_sql` function. Executes the loaded model using beam search and extracts both the generated sequence and its confidence score.
`schema/schema_loader.py`	Connects to the SQLite database and dynamically extracts tables, columns, primary keys, and foreign keys.
`schema/schema_linker.py`	Evaluates the user's natural language question against the database schema to prune irrelevant tables, ensuring the AI model isn't overwhelmed by massive schemas.
`schema/prompt_builder.py`	Formats the pruned schema and the user's question into the exact string format expected by the Spider-trained T5 model (`<question> | <db_id> | <schema>`).
`sql/rule_engine.py`	A fast, zero-cost regex-based engine that attempts to solve extremely simple queries without invoking the ML model.
`sql/sql_executor.py`	Safely executes the final SQL query against the SQLite file and fetches the resulting columns and rows.
`sql/sql_validator.py`	A security gatekeeper. Rejects queries containing `DROP` , `DELETE` , `UPDATE` , or `INSERT` to prevent malicious database modification.
`sql/sql_verifier.py`	Checks if the generated SQL explicitly maps to the known tables and columns in the extracted schema.
`sql/sql_repair.py`	Attempts to fix minor SQL syntax errors or hallucinated column names before execution.
`utils/response_formatter.py`	Standardizes the JSON payload returned to the frontend for both successful queries and pipeline errors.

---

4. The "Life of a Request" (Data Flow)

When a user types "Show me all departments" into the Chat UI, the following sequence occurs:

1. **Frontend Dispatch:** `ChatComponent.jsx` sends a `POST` request with the JSON payload `{"question": "Show me all departments"}` to `/api/query` .
2. **API Entry (`app.py`):** The Flask route extracts the question and verifies the database path configuration.
3. **Schema Extraction (`schema_loader.py`):** The system connects to the active SQLite database and reads its full structural metadata.
4. **Rule Engine (`rule_engine.py`):** The system attempts to match the question against hardcoded templates. If it fails, it moves to the AI pipeline.
5. **Schema Pruning (`schema_linker.py`):** The question is analyzed against the schema to filter out completely irrelevant tables (e.g., ignoring `enrollments` if the question only asks about `departments`).
6. **Prompt Construction (`prompt_builder.py`):** The pruned schema and question are serialized into the T5 prompt format.
7. **Primary Inference (`inference.py`):** The T5 model performs a beam search, returning a raw SQL string and a confidence score.
8. **Fallback Evaluation (`app.py`):** If the T5 model fails or its confidence score is `< -0.5` , the system falls back to the Gemini API (`google-genai`) using the same pruned schema.
9. **Sanitization & Validation (`sql_postprocess.py` , `sql_validator.py`):** The SQL is cleaned of markdown artifacts and checked for destructive commands.
10. **Execution (`sql_executor.py`):** The SQL is run against the database.
11. **Execution Repair (`app.py`):** If the SQL fails at execution (e.g., SQLite syntax error), the error message, schema, and original question are sent to the Gemini API for an automated repair attempt.
12. **Response Formulation (`response_formatter.py`):** The fetched data rows and the successful SQL query are formatted into a standard JSON payload and returned to the frontend.
13. **Frontend Render:** `ChatComponent.jsx` animates the response, rendering the SQL in a code block and the data in a formatted HTML table.

---

5. Core Logic Explanations

AI Model Loading & Inference

The application uses a three-tier loading strategy defined in `load_model.py`. When the server starts, it attempts to load the T5 model from Hugging Face using `.safetensors` to prevent security vulnerabilities associated with `torch.load()`. If unavailable, it falls back to standard `.bin` weights. During inference (`inference.py`), the model utilizes **Beam Search** (`num_beams=4`). Instead of greedily picking the most likely next word, it explores the 4 most promising SQL sequences simultaneously, returning the one with the highest overall probability. The sequence score is returned to `app.py` to dictate whether the Gemini fallback is required.

Validation & Automated Repair

The system employs multiple safety nets:

- **Validation (`sql_validator.py`):** Prevents SQL injection or data loss by strictly blocking DML (Data Manipulation Language) commands. Only `SELECT` statements pass the gate.
- **Pre-execution Repair (`sql_repair.py`):** If the model hallucinates a table name (e.g., `dept` instead of `departments`), the repair module checks the schema and attempts string replacement.
- **Execution Repair (Gemini Fallback):** If SQLite throws an execution error, the exact error trace is passed to the Gemini API with a prompt to "Generate a corrected, valid SQL query." This creates a highly resilient self-healing loop.

Dynamic Database Schemas

Instead of hardcoding databases, `schema_loader.py` dynamically queries SQLite's internal master table (`sqlite_master`) and `PRAGMA table_info()` whenever a file is uploaded. This extracts all table names, column names, data types, and primary/foreign key constraints in real-time. This JSON schema is then injected into the prompt, allowing the AI to understand any database instantly without prior training.

---

6. Scalability & Bottlenecks

Bottlenecks

1. **RAM/VRAM Usage (T5-Large):** The primary bottleneck is the `tscholak/3vnuv1vf` model. T5-Large has ~770M parameters. When loaded into memory, it consumes around 3GB to 4GB of RAM. Concurrent user requests could quickly exhaust memory if the model instance isn't shared or batched properly.
2. **Schema Linking Overhead:** For massive enterprise databases with hundreds of tables, the `schema_linker.py` may become slow, and the serialized schema could exceed the `MAX_INPUT_LENGTH` (512 tokens) of the T5 model.
3. **Synchronous API Calls:** The Flask backend currently processes requests synchronously. Long-running model inferences or Gemini API network calls will block the server thread, reducing throughput for simultaneous users.

Scalability Solutions

1. **Session Management:** Currently, `config.settings.DB_PATH` is a global variable. To support multiple concurrent users querying different databases, the uploaded databases must be tied to user session IDs or JWTs, passing the specific `DB_PATH` into the query pipeline per request.
2. **Asynchronous Processing:** Migrating from Flask to an asynchronous framework like FastAPI (or using Celery/Redis for background jobs) would allow the server to handle high concurrency without blocking threads during model inference.
3. **Vector Search for Schemas:** Instead of regex-based schema pruning, implementing a Vector Database (like ChromaDB or FAISS) to embed and retrieve only the most semantically relevant tables based on the user's question would solve the token limit bottleneck for massive databases.